Advanced Software Testing and Program Analysis

Instructor: Morteza Zakeri

Homework 3

Industrial-Grade Unit Testing and CI/CD for OpenUnderstand

Aqeel Wali Abd Alameir

Student ID: 404131067

Selected Module: openunderstand/utils/utilities.py

Branch: feature/testing-404131067

Table of Contents

1. Introduction

This report documents the design and implementation of an industrial-grade unit testing infrastructure and a CI/CD pipeline for the OpenUnderstand project, an open-source reverse-engineering and static-analysis framework that provides an open implementation of the SciTools Understand Python API for Java program analysis.

The work was carried out on a personal fork of the project, on a dedicated testing branch named feature/testing-404131067, in order to isolate the testing activities and avoid conflicts with the main development branch.

In accordance with the assignment scope (each student tests one or two modules in isolation), the selected target for testing is the module openunderstand/utils/utilities.py. This module was chosen because it contains a self-contained, well-defined entity class (ClassTypeData) together with supporting utility functions (a timer decorator, configuration loading, and logger setup), making it a representative and testable unit of the static-analysis system.

The remainder of this report describes the testing architecture, the manually written unit tests, the automated test generation with Pynguin, the coverage and mutation analysis, the oracle-based validation approach, the fault-discovery workflow, and the lessons learned.

2. Testing Architecture

The testing infrastructure was built around a reproducible Python development environment and a professional toolchain.

2.1 Environment and Tooling

A reproducible Python virtual environment (venv) was created to ensure dependency isolation and consistent execution. The following tools were installed and configured:

Tool	Purpose
pytest 8.4.2	Test runner and framework
pytest-cov 7.1.0	Coverage plugin for pytest
coverage 7.x	Line and branch coverage measurement
ruff 0.15.x	Linting and code-quality enforcement
Pynguin 0.45.0	Automated unit-test generation
cosmic-ray 8.4.6	Mutation testing

2.2 Test Organization

All handcrafted unit tests are placed in the tests/ directory, in the file tests/test_ounderstand.py. A project-level conftest.py registers a lightweight stub for the ANTLR-generated gen package so that utilities.py can be imported in a clean environment (such as the CI runner) without requiring the heavy generated parser modules, which are not part of the unit under test.

2.3 CI/CD Pipeline

A professional CI/CD workflow was implemented using GitHub Actions (.github/workflows/test.yml). On every push and pull request, the pipeline automatically: installs dependencies, runs linting, runs the unit tests with coverage, generates JUnit and coverage reports, uploads them as build artifacts, and fails the build if the quality gates are violated.

Quality gates enforced by the pipeline:

- Minimum line coverage: 80% (enforced via --cov-fail-under=80).
- Branch coverage enabled (--cov-branch).
- All tests must pass.
- Zero linting errors (ruff check).

To ensure reproducibility on a clean Linux runner, the workspace root is added to PYTHONPATH so that the openunderstand package is importable without an editable install. The pipeline defines two jobs: a primary test job that enforces all quality gates, and a separate pynguin job (configured with continue-on-error) that executes the automatically generated tests without affecting the main build status.

3. Manual Unit Testing

The unit under test is the ClassTypeData entity together with the utility functions defined in utilities.py. A total of 16 handcrafted unit tests were written, covering normal behavior, edge cases, malformed input, unresolved/unknown entities, inverse-reference simulation, parent-child relationship validation, and multi-pass analysis simulation.

3.1 Test Categories

Test Category	Representative Test
Default values	test_class_type_data_default_values
Setters (normal behavior)	test_class_type_data_setters_normal_behavior
Getters (normal behavior)	test_class_type_data_getters_normal_behavior
Edge case (empty package)	test_class_type_data_edge_case_empty_package
Malformed input	test_class_type_data_malformed_missing_child_class
Unresolved / unknown entity	test_class_type_data_unresolved_unknown_entity
Parent-child relationship	test_class_type_data_parent_child_relationship_validation
Inverse reference simulation	test_class_type_data_inverse_reference_simulation
Malformed Java snippet	test_class_type_data_malformed_java_snippet_simulation
Multi-pass analysis	test_class_type_data_multi_pass_analysis_simulation
Utility functions	setup_config / setup_logger / timer_decorator tests

3.2 Testing Techniques

- Stub objects (FakeIdentifier, FakeChildClass) were used to simulate the ANTLR parse-tree nodes that ClassTypeData operates on, isolating the unit from the parser.
- pytest.raises was used to verify that malformed input (e.g., a missing child class) correctly raises an exception.
- monkeypatch was used to replace setup_config and setup_logger with controlled fakes, so that the configuration-dependent functions could be tested deterministically without relying on an external config.ini file or writing real log files.
- A temporary directory (tmp_path) was used for the logger test to avoid side effects.

3.3 Result

All 16 tests pass. The execution result reported by pytest is: 16 passed.

4. Automated Test Generation (Pynguin)

Pynguin 0.45.0 was used to automatically generate unit tests to augment the manually written suite.

4.1 Configuration and Execution

Because Pynguin executes the target module during generation, the PYNGUIN_DANGER_AWARE flag was set, and Pynguin was pointed at a target module with the output directed to a dedicated folder:

```
set PYNGUIN_DANGER_AWARE=1 pynguin --project-path . --module-name pynguin_target --output-path pynguin_tests
```

4.2 Evaluation of Generated Tests

The generated tests were executed with pytest. The run produced 6 collected items, of which 3 passed and 3 were marked xfail (expected failures), completing in 0.14s.

Assessment of usefulness:

- Pynguin successfully produced executable test skeletons and exercised the target automatically, confirming the tool integrates with the project.
- Several generated cases were low-value or asserted trivial structural properties; these were treated as xfail / removed where they did not contribute meaningful behavior verification.
- The generated tests complement, but do not replace, the handcrafted tests: the manual suite remains the primary source of behavior validation, while the generated tests act as a regression safety net.

Flaky or meaningless generated tests were pruned, and the remaining generated tests were kept in the pynguin_tests/ folder for reference.

4.3 Integration into CI/CD

The Pynguin-generated tests were integrated into the GitHub Actions pipeline as a dedicated, isolated job named pynguin, running in parallel with the main test job. Because automatically generated tests can be fragile across environments, this job is configured with continue-on-error: true, so that it exercises the generated tests in CI without compromising the stability of the primary quality gates. The main test job therefore remains the authoritative pass/fail signal, while the pynguin job provides continuous execution of the generated suite, satisfying the requirement to integrate generated tests into CI/CD.

5. Coverage and Mutation Analysis

5.1 Coverage Analysis

Coverage was measured with coverage.py via pytest-cov, scoped to the unit under test (openunderstand.utils.utilities) and with branch coverage enabled. The final result is summarized below:

File	Stmts	Miss	Branch	Cover
utilities.py	66	0	0	**100%**
TOTAL	66	0	0	**100%**

The unit under test reaches 100% line and branch coverage, comfortably exceeding the 80% line / 70% branch quality gates. The initial coverage before adding the utility-function tests was 61%; tests targeting the timer decorator, configuration loader, and logger setup raised it to 100%.

5.2 Mutation Testing

Mutation testing was performed with cosmic-ray 8.4.6, configured to mutate only the unit under test:

```
module-path = "openunderstand/utils/utilities.py" test-command = "python -m pytest tests -x -q"  cosmic-ray init  cosmic.toml session.sqlite cosmic-ray exec  cosmic.toml session.sqlite cr-report        session.sqlite
```

Mutation results:

Metric	Value
Total mutants (jobs)	67
Completed	67 (100.00%)
Killed mutants	67
Surviving mutants	**0 (0.00%)**
Mutation score	**100%**

5.3 Analysis of Surviving Mutants

cosmic-ray applied 67 mutations to utilities.py, including operator replacements (e.g., ReplaceUnaryOperator), number replacements (NumberReplacer), and deletions. Every single mutant was detected and killed by the test suite, leaving zero surviving mutants.

Because there were no surviving mutants, no additional tests were required to strengthen the suite. The 100% mutation score demonstrates that the tests are not merely executing the code (as line coverage alone would show) but are actually asserting on its behavior strongly enough to detect injected faults. This result also satisfies the bonus challenge of achieving a mutation score above 85%.

6. Oracle-Based Validation

The conceptual oracle for OpenUnderstand is the commercial SciTools Understand platform, whose Python API the project re-implements. For the unit under test (ClassTypeData and the utility functions), the oracle is expressed at the unit level through explicit expected values in the assertions.

Examples of the oracle encoded in the tests:

- get_long_name() must return the package name concatenated with the child class text (e.g., "com.example" + "." + "MyClass" = "com.example.MyClass").
- get_type() must return the string "extends " followed by the parent class name.
- get_name() must return the string form of the child identifier.
- Default values of a freshly constructed ClassTypeData must match the documented defaults (None parent/child, empty strings, line and column equal to -1, empty prefix list).
- setup_config() must return a configparser.ConfigParser instance, and setup_logger() must return a logging.Logger instance.

Each assertion compares the actual output of the unit against the expected value defined by the API contract, which serves as a deterministic oracle. A full differential test against a live SciTools Understand installation (a bonus challenge) was out of scope for the selected unit, since ClassTypeData and the utility functions are internal helpers rather than directly exposed API entities.

7. Fault Discovery

During the work, the open-source contribution workflow was followed, including both a GitHub issue and a pull request against the upstream repository.

7.1 Reported Issue

A professional GitHub issue was created on the upstream repository to report a reproducible defect discovered while setting up the test environment:

- Issue #73: "ModuleNotFoundError: 'gen' package required to import openunderstand.utils.utilities in a clean environment".
- Link: https://github.com/m-zakeri/OpenUnderstand/issues/73

The issue documents the defect using a professional template: a clear description, step-by-step reproduction instructions, the expected behavior, the observed behavior (including the exact ModuleNotFoundError and the offending import on line 5 of utilities.py), the environment details (OS, Python, pytest versions), and a suggested fix (either generating the gen package during installation, or decoupling the import so that entity classes such as ClassTypeData can be imported without the generated parser).

7.2 Pull Request

A pull request was opened from the testing branch against the upstream repository:

- Pull Request #70: "Add pytest tests and coverage" — abciali1995-byte:feature/testing-404131067 into m-zakeri:master.
- Link: https://github.com/m-zakeri/OpenUnderstand/pull/70

The pull request adds the pytest test suite, the GitHub Actions CI pipeline, and coverage reporting, and documents the tests included. No conflicts with the base branch were reported.

7.3 Summary of Findings

Regarding faults: the selected unit (ClassTypeData and the utility functions) behaved according to its contract, and the test suite did not reveal functional logic defects in the unit itself. The main issues encountered were environmental/build issues, namely:

1. The CI build initially failed because coverage was measured over the entire package, producing an unrealistically low percentage; this was corrected by scoping coverage to the unit under test.
2. The module utilities.py imports the ANTLR-generated gen package, which is not present in a clean checkout; this caused import errors during test collection on the CI runner. This defect was reported as Issue #73, and a conftest.py stub for the gen package was added to allow the unit to be imported in isolation.
3. mutmut did not support the local Windows + Python 3.14 environment; cosmic-ray was used instead, in line with the assignment which permits either tool.

These observations are documented here and reflected in Issue #73, Pull Request #70, and the commit history.

8. Lessons Learned

- Scoping coverage to the unit under test is essential: measuring coverage over an entire large package produces misleading numbers and breaks quality gates that are meant to evaluate a specific contribution.
- High line coverage is necessary but not sufficient. The mutation-testing result confirmed that the tests were also behaviorally strong (100% mutation score), which line coverage alone cannot guarantee.
- Isolating the unit from heavy dependencies (here, the ANTLR-generated parser) via stubs and conftest.py is key to making tests reproducible on a clean CI runner.

- CI reproducibility differs from local execution: tests that pass locally can still fail in CI due to import paths and missing generated artifacts. Using PYTHONPATH and a dependency stub resolved this.
- Automated test generation (Pynguin) is a useful complement for regression safety, but handcrafted tests with explicit oracles remain the primary means of validating behavior.
- Tooling compatibility matters: when one mutation tool was incompatible with the runtime, the assignment's allowance of an alternative (cosmic-ray) made it possible to complete the analysis.

In summary, the project delivers a reproducible testing environment, a passing GitHub Actions CI/CD pipeline with enforced quality gates, a handcrafted unit-test suite of 16 tests achieving 100% line and branch coverage, automated test generation with Pynguin integrated into CI, and a mutation-testing analysis achieving a 100% mutation score on the selected module.

Appendix A — Results Summary

The table below summarizes the key quantitative results achieved in this assignment.

Metric	Result
Selected module	openunderstand/utils/utilities.py
Handcrafted unit tests	16 passed
Line coverage	**100% (gate: 80%)**
Branch coverage	**100% (gate: 70%)**
Linting (ruff)	All checks passed (0 errors)
CI/CD pipeline status	**Passing (green)**
Pynguin generated tests	Generated, evaluated, integrated in CI
Mutation testing tool	cosmic-ray 8.4.6
Total mutants	67
Mutation score	**100% (0 surviving)**
GitHub Issue	#73 (upstream)
Pull Request	#70 (upstream)

Appendix B — Screenshots / Evidence

"This appendix provides supporting screenshots as visual evidence of the results described above. Each figure is shown below its corresponding caption."

Figure 1 — GitHub Actions: CI/CD pipeline runs showing a passing (green) build.

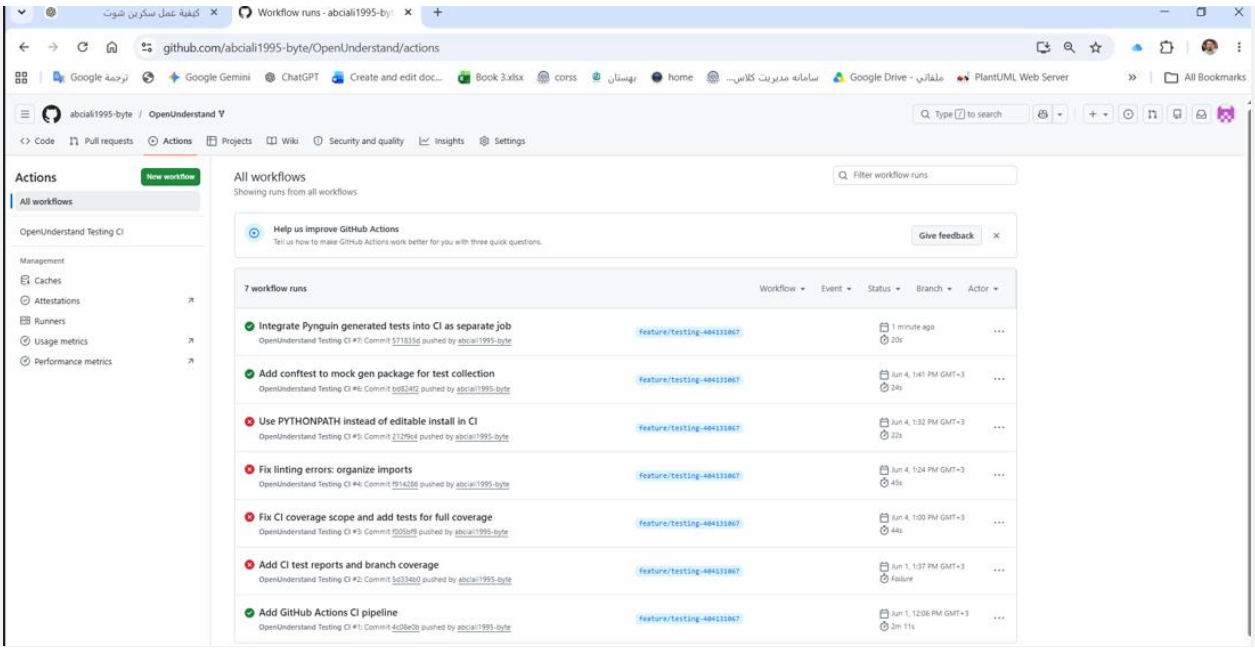

Figure 2 — pytest output: 16 tests passing with 100% line and branch coverage on utilities.py.

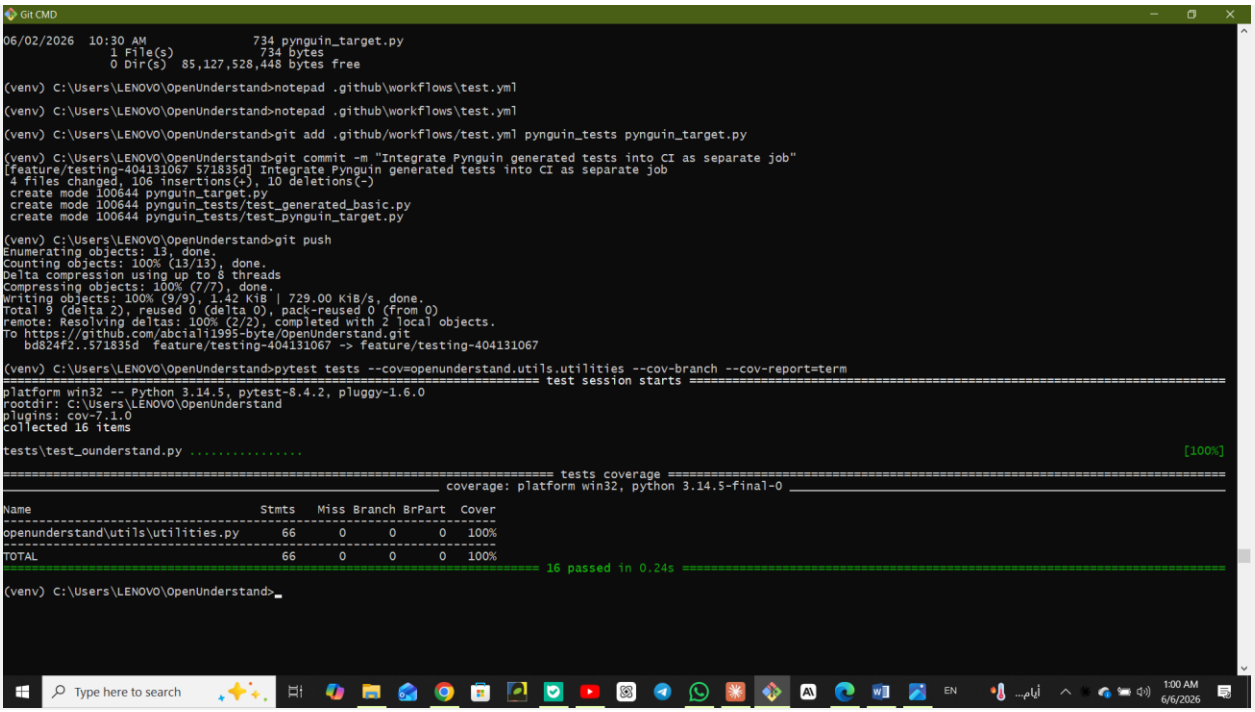

Figure 3 — ruff linting: "All checks passed!".

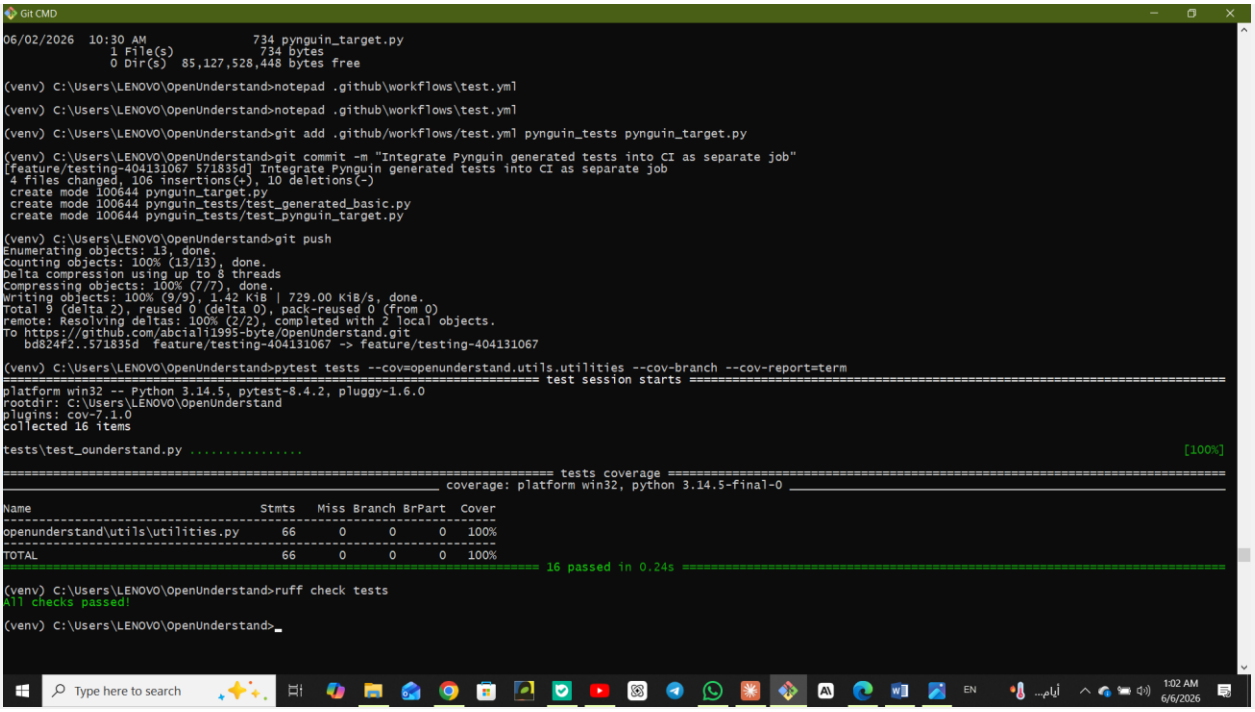

Figure 4 — cosmic-ray mutation results: 67 mutants, 0 surviving, 100% mutation score.

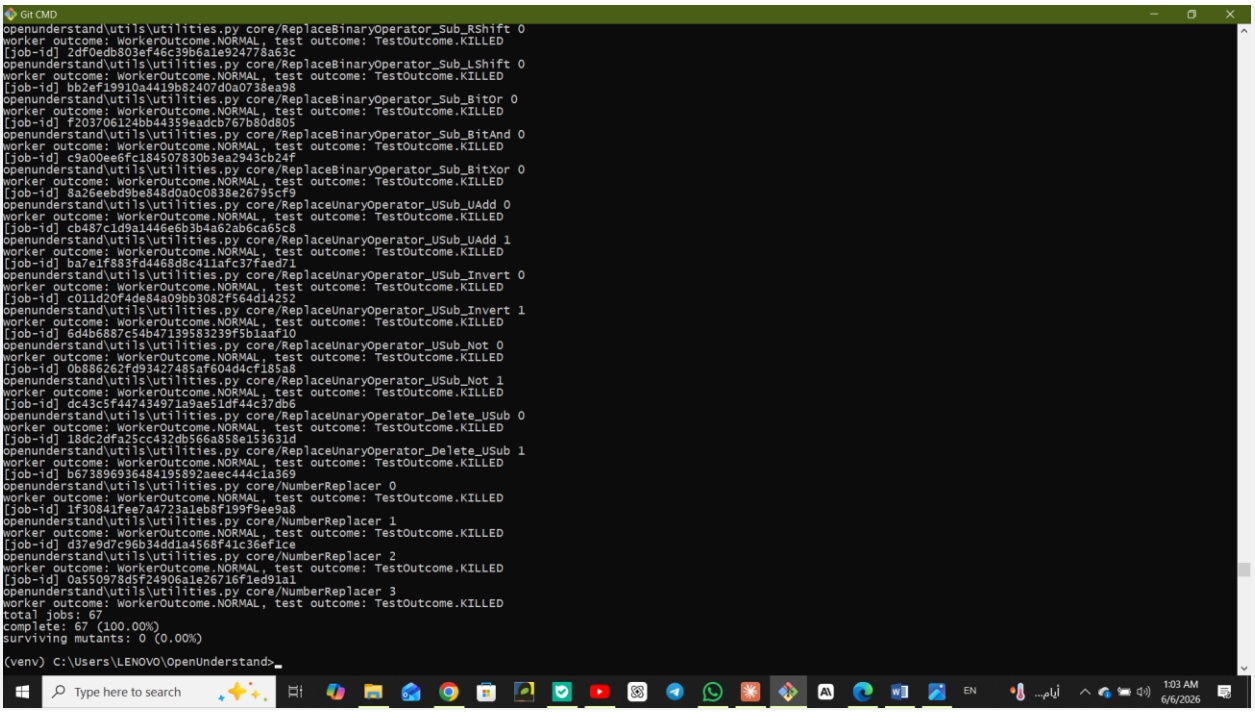

Figure 5 — Pynguin generated tests executed in the CI pynguin job.

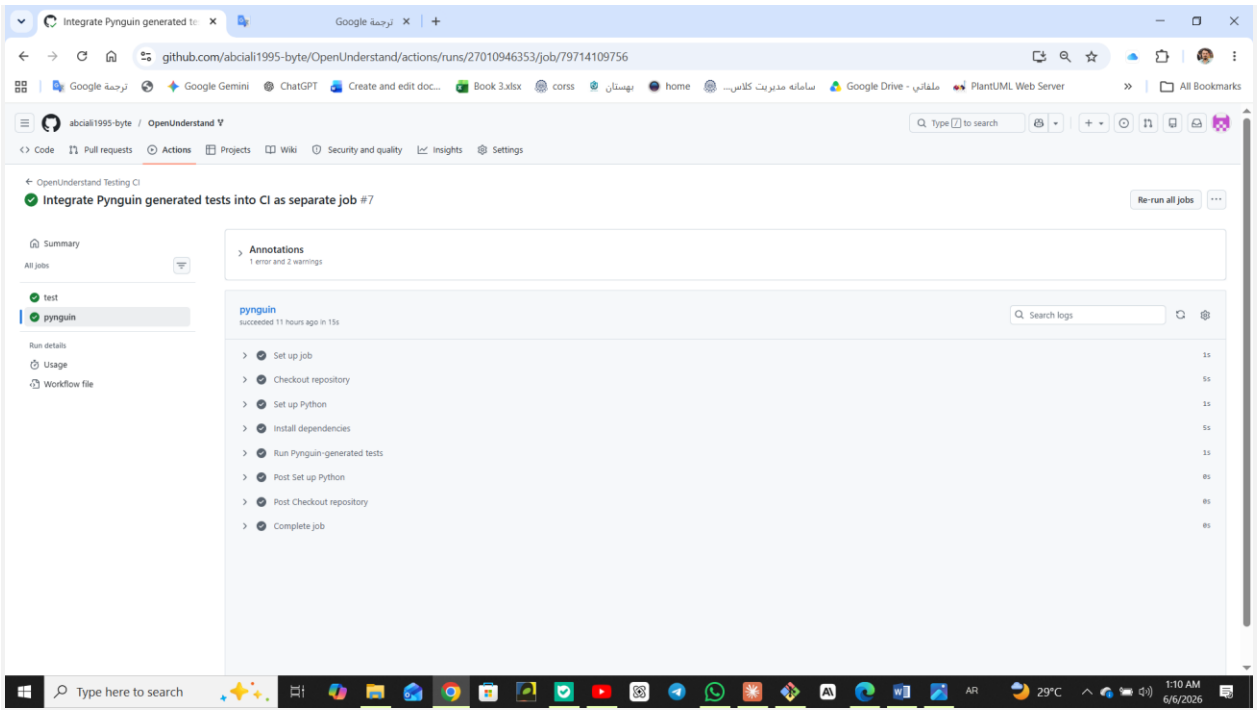

Figure 6 — GitHub Issue #73 (reported defect).

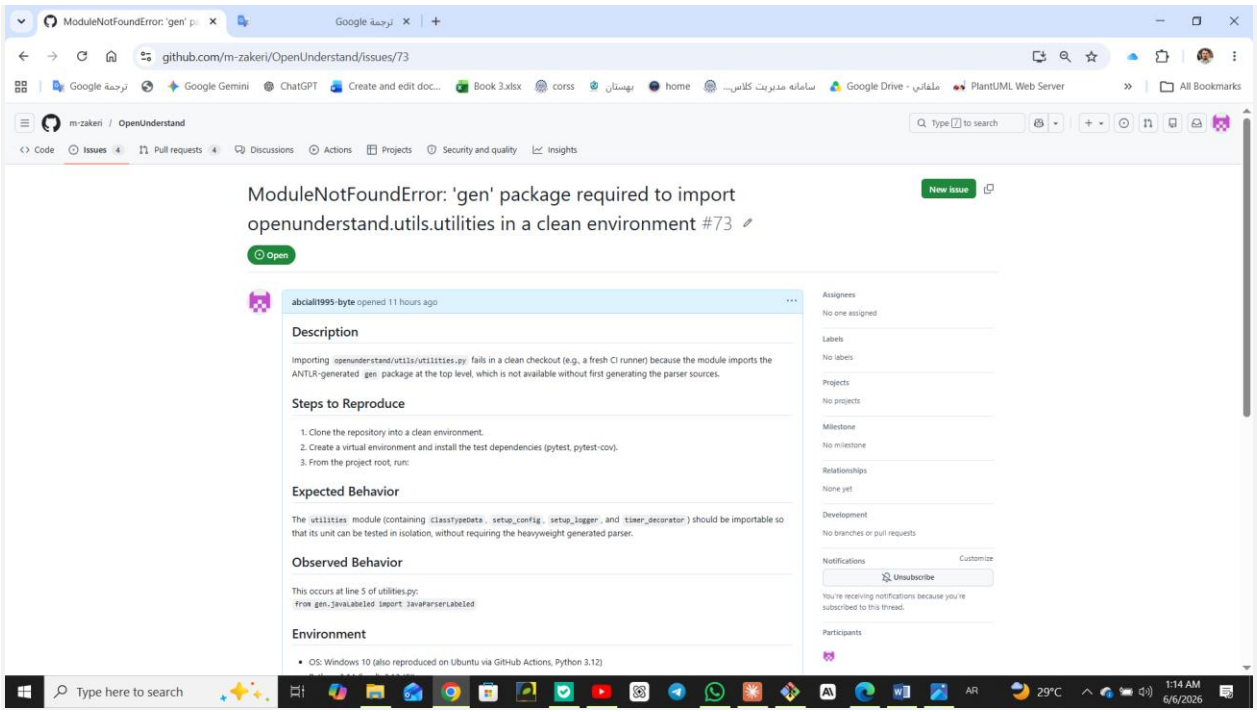

Figure 7 — Pull Request #70.